Plantilla de Informe de Trabajo Práctico

1. Carátula

- Título del Trabajo: Trabajo práctico N°2
- Integrantes del equipo:
  - Hasenauer, Fernanda.
  - Hernandez, Tomas.
  - Leguizamon, Augusto
- Fecha de entrega: 02/11/2026
- Link del repositorio: https://github.com/Hernandez-TV/AyED2026c2-Hernandez-Hasenauer-Leguizamon/tree/main

2. Introducción

En este trabajo práctico se abordaron distintos problemas utilizando estructuras de datos avanzadas y algoritmos eficientes para resolver situaciones reales.

El primer problema consistió en desarrollar un sistema de gestión de pacientes para una sala de emergencias, priorizando la atención según el nivel de riesgo mediante una cola de prioridad implementada con un montículo binario.

El segundo problema consistió en implementar una base de datos de temperaturas utilizando un árbol AVL. El objetivo fue permitir búsquedas, inserciones y consultas por rango de fechas de manera eficiente.

Finalmente, el tercer problema abordó una situación de logística y transporte modelada mediante grafos dirigidos ponderados, permitiendo determinar capacidades máximas de transporte y costos mínimos de traslado entre ciudades.

Durante el desarrollo se aplicaron conceptos de árboles, montículos binarios, árboles AVL, colas de prioridad, grafos y algoritmos de búsqueda sobre grafos.

3. Desarrollo de los problemas

3.1 Problema N° 1: Sala de Emergencias

A. Análisis y planteo

El problema consiste en administrar el ingreso y atención de pacientes en una sala de emergencias. La principal dificultad radica en que el orden de atención no depende del orden de llegada sino del nivel de criticidad de cada paciente.

Las operaciones principales son:

- Ingreso de pacientes.
- Obtención del paciente más crítico.
- Eliminación del paciente atendido.

- Desempate entre pacientes con igual criticidad.

Para resolverlo se utilizó una cola de prioridad implementada mediante un montículo binario de mínimos.

Esta decisión resulta adecuada porque:

- El paciente más grave posee prioridad máxima.
- Las inserciones poseen complejidad O(log n).
- La atención del siguiente paciente también posee complejidad O(log n).

Para resolver empates se agregó un criterio de orden de llegada, garantizando que pacientes con la misma criticidad sean atendidos respetando el orden temporal de ingreso.

B. Implementación / Resolución

La solución se dividió en los siguientes módulos:

- Paciente
- MonticuloBinario
- ColaDePrioridad
- SalaDeEmergencia
- Main

El montículo binario almacena objetos Paciente y utiliza como criterio de comparación:

(riesgo, orden_llegada)

De esta forma se priorizan los pacientes con menor nivel de riesgo numérico y, en caso de empate, aquellos que ingresaron primero.

La cola de prioridad encapsula el funcionamiento interno del montículo y la sala de emergencias administra el ingreso y atención de pacientes.

C. Resultados y pruebas

Se verificó:

- Inserción correcta de pacientes.
- Atención en orden de prioridad.
- Correcto desempate por orden de llegada.
- Funcionamiento de la simulación proporcionada por la cátedra.

3.2 Problema N° 2: Temperaturas_DB

A. Análisis y planteo

El objetivo consistió en implementar una base de datos de temperaturas utilizando un árbol AVL.
Las operaciones solicitadas incluyen:

- Inserción.
- Consulta por fecha.

- Eliminación.
- Búsqueda de temperaturas mínimas y máximas.
- Consultas por rango de fechas.

Las operaciones de búsqueda e inserción son críticas, por lo que se seleccionó un árbol AVL.
La elección es razonable porque mantiene el árbol balanceado automáticamente asegurando complejidad O(log n) para inserción, eliminación y búsqueda.

B. Implementación / Resolución

La implementación se dividió en:

- NodoAVL
- ArbolAVL
- Temperaturas_DB

El AVL almacena como clave un objeto datetime correspondiente a la fecha de registro y como valor la temperatura asociada.
Se implementaron:

- guardar_temperatura()
- devolver_temperatura()
- devolver_temperaturas()
- max_temp_rango()
- min_temp_rango()
- temp_extremos_rango()
- borrar_temperatura()
- cantidad_muestras()

Para mantener el balance del árbol se utilizaron:

- Rotación simple izquierda.
- Rotación simple derecha.
- Rotación doble izquierda-derecha.
- Rotación doble derecha-izquierda.

C. Resultados y pruebas

Se realizaron pruebas de:

- Inserción de registros.
- Consultas individuales.
- Consultas por rango.
- Obtención de extremos.
- Eliminación de fechas.
- Rebalanceo del AVL.

3.3 Problema N° 3: Servicio de Transporte

A. Análisis y planteo

El problema describe una red de ciudades conectadas mediante rutas de transporte.
Cada ruta posee:

- Ciudad origen.
- Ciudad destino.
- Capacidad máxima de carga.

- Precio.

La estructura natural para modelar esta situación es un grafo dirigido.
Las operaciones requeridas fueron:

1. Encontrar el máximo cuello de botella entre Buenos Aires y una ciudad destino.
2. Encontrar el costo mínimo de transporte para un peso determinado.

B. Implementación / Resolución

La solución se desarrolló mediante:

- Grafo
- Ruta
- ServicioTransporte
- LectorArchivos

Para el primer punto se implementó una variante del problema del máximo cuello de botella. La capacidad de un camino está determinada por la arista con menor capacidad dentro de dicho recorrido.
La solución busca maximizar ese valor.
Para el segundo punto se utilizó un algoritmo similar a Dijkstra, considerando únicamente las rutas cuya capacidad permite transportar el peso solicitado y minimizando el costo total del recorrido.

Análisis de complejidad:

Método	**Complejidad**
guardar_temperatura()	O(log n)
devolver_temperatura()	O(log n)
borrar_temperatura()	O(log n)
cantidad_muestras()	O(1)
devolver_temperaturas()	O(n)
max_temp_rango()	O(n)
min_temp_rango()	O(n)
temp_extremos_rango()	O(n)

Justificación:

Las operaciones guardar_temperatura(), devolver_temperatura() y borrar_temperatura() se implementan sobre un árbol AVL. Debido a que el AVL mantiene su altura balanceada mediante rotaciones, la altura del árbol es proporcional a log(n). Por esta razón, las operaciones de inserción, búsqueda y eliminación poseen complejidad temporal O(log n).

La operación cantidad_muestras() únicamente devuelve un contador almacenado en la estructura, por lo que su complejidad es constante O(1).

Por otro lado, los métodos devolver_temperaturas(), max_temp_rango(), min_temp_rango() y temp_extremos_rango() requieren recorrer los elementos comprendidos dentro de un rango de fechas. En el peor caso, la consulta puede involucrar todos los nodos almacenados en la estructura, resultando en una complejidad O(n).

C. Resultados y pruebas

Se verificó:

- Carga correcta del archivo de rutas.
- Determinación del cuello de botella máximo.
- Obtención del costo mínimo según el peso solicitado.
- Correcta representación del grafo dirigido.

4. Análisis de resultados específicos / Discusión

Durante el desarrollo del trabajo práctico se observó que la elección de la estructura de datos tiene un impacto directo sobre la eficiencia de la solución obtenida para cada problema.

En el Problema 1 se seleccionó una cola de prioridad implementada mediante un montículo binario de mínimos debido a que la operación más importante consiste en identificar rápidamente al paciente con mayor prioridad de atención. A diferencia de una cola convencional, donde los pacientes serían atendidos únicamente por orden de llegada, el uso del montículo permite garantizar que el paciente con mayor criticidad se encuentre siempre accesible en la raíz de la estructura. Además, la incorporación del orden de llegada como segundo criterio permite resolver correctamente los casos en que varios pacientes compartían el mismo nivel de criticidad.

En el Problema 2 se utilizó un árbol AVL para almacenar las mediciones de temperatura indexadas por fecha. La principal ventaja observada fue la capacidad de mantener el árbol balanceado automáticamente luego de cada inserción o eliminación. Esto permitió conservar una complejidad logarítmica para las operaciones de búsqueda, inserción y borrado independientemente del orden en que se agregaran los datos. Las consultas por rango de fechas también resultaron adecuadas para este tipo de estructura, ya que pueden aprovechar el recorrido ordenado del árbol para recuperar únicamente la información relevante.

Por otra parte, la implementación de las rotaciones simples y dobles constituyó uno de los aspectos más importantes de este problema, ya que son las responsables de garantizar el equilibrio del árbol y evitar que se degrade hacia una estructura lineal. Durante las pruebas realizadas se verificó que las operaciones de búsqueda mantuvieron tiempos de respuesta consistentes aun cuando la cantidad de registros aumentó.

En el Problema 3 se modeló la red de transporte mediante un grafo dirigido. Esta representación resultó natural debido a que las rutas entre ciudades poseen una dirección específica y características particulares asociadas, como capacidad máxima de carga y costo de transporte. El primer requerimiento se resolvió mediante un algoritmo orientado a

maximizar el cuello de botella de cada recorrido, mientras que el segundo se abordó mediante una estrategia de búsqueda de costo mínimo considerando únicamente aquellas rutas capaces de transportar el peso solicitado.

Comparando los tres problemas, puede observarse que cada uno exige estructuras de datos diferentes debido a la naturaleza de las operaciones predominantes. Mientras que la cola de prioridad optimiza la selección de elementos según su urgencia, el árbol AVL resulta especialmente adecuado para realizar búsquedas ordenadas y consultas eficientes sobre grandes conjuntos de datos. Por otro lado, los grafos permiten representar relaciones complejas entre entidades y resolver problemas de conectividad, capacidad y optimización de recorridos.

En conclusión, los resultados obtenidos muestran que la selección apropiada de la estructura de datos fue un aspecto fundamental para alcanzar soluciones eficientes en cada uno de los escenarios planteados. La implementación permitió aplicar de manera práctica conceptos vistos durante la cursada y comprender la relación existente entre los requerimientos de un problema y las estructuras más convenientes para resolverlo.

5. Declaración de Uso Ético de Inteligencia Artificial

Herramienta utilizada: Microsoft Copilot.
Descripción de uso: generación de ideas, estructuras. Se utilizó para discutir alternativas de diseño y organización de módulos.

Declaración de responsabilidad:

Los integrantes del grupo declaramos que hemos revisado y validado personalmente toda la información y código sugerido por las herramientas de IA. Entendemos los conceptos presentados en este informe y asumimos la responsabilidad académica por la integridad y veracidad del contenido. El trabajo final es producto de nuestro propio análisis y criterio.

6. Trabajo en equipo

Estimación inicial:

- Análisis de consignas: 3 horas.
- Implementación Proyecto 1: 4 horas.
- Implementación Proyecto 2: 6 horas.
- Implementación Proyecto 3: 5 horas.
- Pruebas y documentación: 3 horas.

Total estimado: 21 horas.

Tiempo real:

La implementación del AVL demandó más tiempo del previsto debido a las rotaciones y operaciones de eliminación. Asimismo, la integración y pruebas del sistema de temperaturas generó tareas adicionales de depuración.

La coordinación se realizó mediante intercambio de mensajes, reuniones virtuales y uso compartido del repositorio GitHub.

La integración de cambios se realizó utilizando Git mediante commits, pull y push periódicos.

7. Conclusiones

Los objetivos planteados fueron alcanzados satisfactoriamente.
El trabajo permitió aplicar distintas estructuras de datos para resolver problemas reales con requisitos de eficiencia diferentes. Se profundizó el uso de montículos binarios, colas de prioridad, árboles AVL y grafos, comprendiendo tanto sus fundamentos teóricos como sus aplicaciones prácticas.
Las principales dificultades estuvieron asociadas al mantenimiento del balance del AVL y a la adaptación de algoritmos de búsqueda sobre grafos a las restricciones específicas del problema del transporte. Estas dificultades fueron superadas mediante pruebas incrementales y validación continua de resultados.

8. Referencias y Bibliografía

- Apuntes de cátedra de Algoritmos y Estructuras de Datos.
- Libro Runestone Academy - Python y Estructuras de Datos.
- Material teórico proporcionado por la cátedra.
- Documentación oficial de Python.
- Repositorio GitHub del Trabajo Práctico.